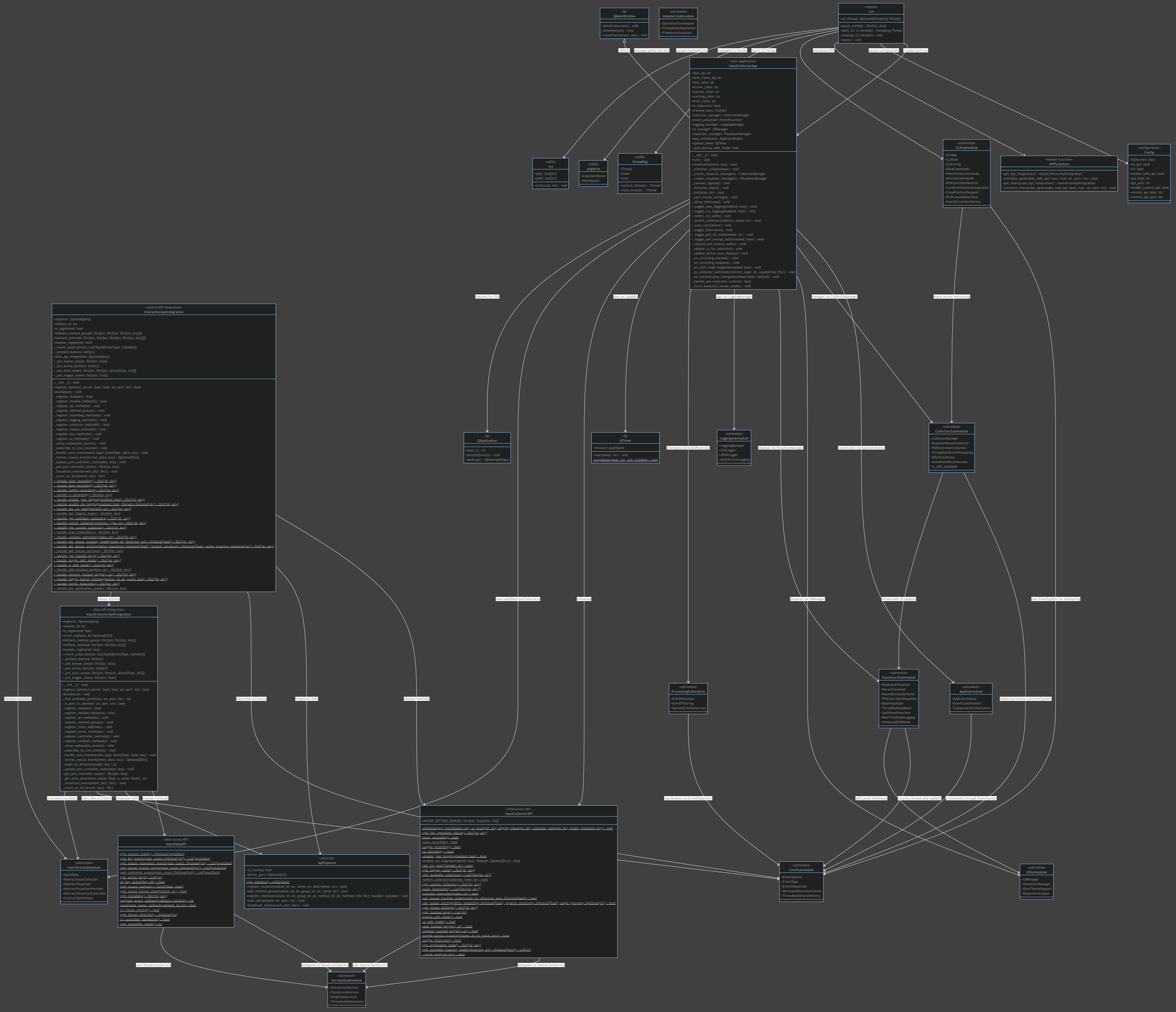
«Qt»
QMainWindow
+showFullScreen() : void
+showNormal() : void
+closeEvent(event: Any) : void
«submodule»
AdaptersSubmodule
+QtInteractionAdapter
+ThreadSafeAdaptation
+FrameworkIsolation
«module»
run
+cli_thread: Optional[threading.Thread]
+parse_config() : Dict[str, Any]
+start_cli_in_thread() : threading.Thread
+cleanup_cli_thread() : void
+main() : void
inherits
manages paths and exit
parses command line
manages CLI thread
starts CLI thread
initializes APIs
parses configuration
creates and runs
«main application»
InputCollectorApp
+dark_bg: str
+dark_frame_bg: str
+text_color: str
+accent_color: str
+success_color: str
+warning_color: str
+error_color: str
+is_fullscreen: bool
+tracked_keys: List[str]
+collector_manager: CollectorManager
+event_processor: EventProcessor
+logging_manager: LoggingManager
+ui_manager: UIManager
+visualizer_manager: VisualizerManager
+app_coordinator: AppCoordinator
+update_timer: QTimer
+_ps4_overlay_edit_mode: bool
+__init__() : void
+run() : void
+closeEvent(event: Any) : void
-_initialize_components() : void
-_create_collector_manager() : CollectorManager
-_create_visualizer_manager() : VisualizerManager
-_connect_signals() : void
-_initialize_state() : void
-_initialize_ui() : void
-_sync_mouse_settings() : void
-_setup_shortcuts() : void
+_toggle_json_logging(enabled: bool) : void
+_toggle_csv_logging(enabled: bool) : void
+_select_csv_path() : void
+_switch_collector(collector_name: str) : void
+_scan_controllers() : void
+_toggle_fullscreen() : void
+_toggle_ps4_viz_mode(mode: str) : void
+_toggle_ps4_overlay_edit(enabled: bool) : void
+_discard_ps4_overlay_edits() : void
-_update_ui_for_collector() : void
-_update_active_keys_display() : void
-_on_recording_started() : void
-_on_recording_stopped() : void
-_on_edit_mode_toggled(enabled: bool) : void
-_on_collector_switched(collector_type: str, capabilities: Dict) : void
-_on_tracked_keys_changed(tracked_keys: List[str]) : void
-_handle_ps4_controller_switch() : bool
-_force_keyboard_mouse_mode() : void
«stdlib»
sys
+argv: List[str]
+path: List[str]
+exit(code: int) : void
«stdlib»
argparse
+ArgumentParser
+Namespace
«stdlib»
threading
+Thread
+Event
+Lock
+current_thread() : Thread
+main_thread() : Thread
«submodule»
CLISubmodule
+CLIApp
+CLIShell
+CLIConfig
+DataCommands
+InteractionCommands
+MonitorCommands
+PS4ControllerMonitor
+CoreEventSystemIntegration
+CrossPlatformSupport
+RichConsoleInterface
+EventDrivenMonitoring
«module functions»
APIFunctions
+get_api_integration() : InputCollectorApiIntegration
+initialize_api(enable_web_api: bool, host: str, port: int) : bool
+get_interaction_api_integration() : InteractionApiIntegration
+initialize_interaction_api(enable_web_api: bool, host: str, port: int) : bool
«configuration»
Config
+fullscreen: bool
+no_gui: bool
+cli: bool
+enable_web_api: bool
+api_host: str
+api_port: int
+enable_control_api: bool
+control_api_host: str
+control_api_port: int
requires for GUI
uses for updates
logs via LoggingManager
manages via CollectorManager
event-driven monitoring
«control API integration»
InteractionApiIntegration
+explorer: Optional[Any]
+module_id: str
+is_registered: bool
+fallback_method_groups: Dict[str, Dict[str, Dict[str, Any]]]
+fallback_methods: Dict[str, Dict[str, Dict[str, Dict[str, Any]]]]
+module_registered: bool
+_event_subscriptions: List[Tuple[EventType, Callable]]
+_pressed_buttons: Set[str]
+data_api_integration: Optional[Any]
+_ps4_button_states: Dict[str, bool]
+_ps4_active_buttons: Set[str]
+_ps4_stick_states: Dict[str, Dict[str, Union[float, str]]]
+_ps4_trigger_states: Dict[str, float]
+__init__() : void
+register_api(start_server: bool, host: str, port: int) : bool
+shutdown() : void
-_register_module() : bool
-_register_module_fallback() : bool
-_register_all_methods() : void
-_register_method_groups() : void
-_register_recording_methods() : void
-_register_logging_methods() : void
-_register_collector_methods() : void
-_register_mouse_methods() : void
-_register_key_methods() : void
-_register_ui_methods() : void
-_setup_websocket_events() : void
-_subscribe_to_core_events() : void
-_handle_core_event(event_type: EventType, data: Any) : void
-_format_mouse_event(event_data: Any) : Optional[Dict]
-_update_ps4_controller_state(data: Any) : void
-_get_ps4_controller_state() : Dict[str, Any]
-_broadcast_event(event_dict: Dict) : void
-_event_to_dict(event: Any) : Dict
+_handle_start_recording() : Dict[str, Any]
+_handle_stop_recording() : Dict[str, Any]
+_handle_toggle_recording() : Dict[str, Any]
+_handle_is_recording() : Dict[str, Any]
+_handle_enable_json_logging(enabled: bool) : Dict[str, Any]
+_handle_enable_csv_logging(enabled: bool, filepath: Optional[str]) : Dict[str, Any]
+_handle_set_csv_path(filepath: str) : Dict[str, Any]
+_handle_get_logging_state() : Dict[str, Any]
+_handle_get_available_collectors() : Dict[str, Any]
+_handle_switch_collector(collector_type: str) : Dict[str, Any]
+_handle_get_current_collector() : Dict[str, Any]
+_handle_scan_controllers() : Dict[str, Any]
+_handle_connect_controller(index: int) : Dict[str, Any]
+_handle_set_mouse_tracking_mode(mode: str, direction_only: Optional[bool]) : Dict[str, Any]
+_handle_set_mouse_settings(delta_smoothing: Optional[float], joystick_sensitivity: Optional[float], angle_grouping: Optional[int]) : Dict[str, Any]
+_handle_get_mouse_settings() : Dict[str, Any]
+_handle_get_tracked_keys() : Dict[str, Any]
+_handle_toggle_edit_mode() : Dict[str, Any]
+_handle_is_edit_mode() : Dict[str, Any]
+_handle_add_tracked_key(key: str) : Dict[str, Any]
+_handle_remove_tracked_key(key: str) : Dict[str, Any]
+_handle_toggle_button_tracking(button_id: str, track: bool) : Dict[str, Any]
+_handle_toggle_fullscreen() : Dict[str, Any]
+_handle_get_application_state() : Dict[str, Any]
«Qt»
QApplication
+exec_() : int
+processEvents() : void
+desktop() : QDesktopWidget
«Qt»
QTimer
+timeout: pyqtSignal
+start(msec: int) : void
+singleShot(msec: int, slot: Callable) : void
processes via EventProcessor
«submodule»
LoggingSubmodule
+LoggingManager
+CSVLogger
+JSONLogger
+MultiformatLogging
displays via VisualizerManager
coordinates via AppCoordinator
«submodule»
CollectorsSubmodule
+CollectorManager
+KeyboardMouseCollector
+PS4ControllerCollector
+ThreadSafeEventProcessing
+SDL2EventLoop
+AutomaticReconnection
+is_sdl2_available
reuses utilities
uses InputData and Interfaces
initializes
presents via UIManager
thread-safe UI updates
uses EventSystem for monitoring
«data API integration»
InputCollectorApiIntegration
+explorer: Optional[Any]
+module_id: str
+is_registered: bool
+event_callback_id: Optional[int]
+fallback_method_groups: Dict[str, Dict[str, Any]]
+fallback_methods: Dict[str, Dict[str, Any]]
+module_registered: bool
+_event_subscriptions: List[Tuple[EventType, Callable]]
+_pressed_buttons: Set[str]
+_ps4_button_states: Dict[str, bool]
+_ps4_active_buttons: Set[str]
+_ps4_stick_states: Dict[str, Dict[str, Union[float, str]]]
+_ps4_trigger_states: Dict[str, float]
+__init__() : void
+register_api(start_server: bool, host: str, port: int) : bool
+shutdown() : void
-_find_available_port(host: str, port: int) : int
-_is_port_in_use(host: str, port: int) : bool
-_register_module() : bool
-_register_module_fallback() : bool
-_register_all_methods() : void
-_register_method_groups() : void
-_register_state_methods() : void
-_register_event_methods() : void
-_register_controller_methods() : void
-_register_callback_methods() : void
-_setup_websocket_events() : void
-_subscribe_to_core_events() : void
-_handle_core_event(event_type: EventType, data: Any) : void
-_format_mouse_event(event_data: Any) : Optional[Dict]
-_angle_to_direction(angle: int) : str
-_update_ps4_controller_state(data: Any) : void
-_get_ps4_controller_state() : Dict[str, Any]
-_get_stick_direction(x_value: float, y_value: float) : str
-_broadcast_event(event_dict: Dict) : void
-_event_to_dict(event: Any) : Dict
handles InputData
subscribes to events
integrates with
exposes methods
«submodule»
ProcessingSubmodule
+EventProcessor
+EventFiltering
+SpecialEventDetection
«submodule»
VisualizersSubmodule
+KeyboardVisualizer
+MouseVisualizer
+MouseButtonsSelector
+PS4ControllerVisualizer
+BaseVisualizer
+ThreadSafeUpdates
+SplitPanelInterface
+RealTimeStateLogging
+EnhancedEditMode
«submodule»
AppSubmodule
+AppCoordinator
+EventCoordination
+ComponentOrchestration
dispatches events via EventSystem
processes InputData
subscribes to events
integrates with
exposes methods
coordinates via EventDispatcher
split panel interfaces
receives thread-safe updates
orchestrates through EventSystem
«interaction API»
InputCollectorAPI
+MOUSE_SETTING_RANGES: Dict[str, Tuple[int, int]]
+initialize(app_coordinator: Any, ui_manager: Any, logging_manager: Any, collector_manager: Any, event_processor: Any) : void
+get_last_operation_status() : Dict[str, Any]
+start_recording() : bool
+stop_recording() : bool
+toggle_recording() : bool
+is_recording() : bool
+enable_json_logging(enabled: bool) : bool
+enable_csv_logging(enabled: bool, filepath: Optional[str]) : bool
+set_csv_path(filepath: str) : bool
+get_logging_state() : Dict[str, Any]
+get_available_collectors() : List[Dict[str, str]]
+switch_collector(collector_type: str) : bool
+get_current_collector() : Dict[str, Any]
+scan_controllers() : List[Dict[str, Any]]
+connect_controller(index: int) : bool
+set_mouse_tracking_mode(mode: str, direction_only: Optional[bool]) : bool
+set_mouse_settings(delta_smoothing: Optional[float], joystick_sensitivity: Optional[float], angle_grouping: Optional[int]) : bool
+get_mouse_settings() : Dict[str, Any]
+get_tracked_keys() : List[str]
+toggle_edit_mode() : bool
+is_edit_mode() : bool
+add_tracked_key(key: str) : bool
+remove_tracked_key(key: str) : bool
+toggle_button_tracking(button_id: str, track: bool) : bool
+toggle_fullscreen() : bool
+get_application_state() : Dict[str, Any]
+get_available_tracking_modes(direction_only: Optional[bool]) : List[str]
-_parse_bool(val: Any) : bool
«data access API»
InputDataAPI
+get_current_event() : Optional[InputData]
+get_key_events(max_count: Optional[int]) : List[InputData]
+get_mouse_movement_events(max_count: Optional[int]) : List[InputData]
+get_mouse_button_events(max_count: Optional[int]) : List[InputData]
+get_controller_events(max_count: Optional[int]) : List[InputData]
+get_active_keys() : List[str]
+is_key_active(key: str) : bool
+get_mouse_position() : Tuple[float, float]
+get_mouse_button_state(button: str) : bool
+get_metadata() : Dict[str, Any]
+register_event_callback(callback: Callable) : int
+unregister_event_callback(callback_id: int) : bool
+is_mouse_moving() : bool
+get_mouse_direction() : Optional[str]
+is_controller_connected() : bool
+get_controller_name() : str
«submodule»
InterfacesSubmodule
+InputData
+AbstractInputCollector
+AbstractVisualizer
+AbstractVisualizerProvider
+AbstractVisualizerController
+ContractDefinitions
«external»
ApiExplorer
+is_running: bool
+server_port: Optional[int]
+get_instance() : ApiExplorer
+register_module(module_id: str, name: str, description: str) : bool
+add_method_group(module_id: str, group_id: str, name: str) : bool
+register_method(module_id: str, group_id: str, method_id: str, method_info: Dict, handler: Callable) : void
+start_server(host: str, port: int) : void
+broadcast_event(event_dict: Dict) : void
«submodule»
CoreSubmodule
+EventSystem
+EventType
+EventDispatcher
+DecoupledCommunication
+ThreadSafeEventDelivery
«submodule»
UISubmodule
+UIManager
+VisualizerManager
+DarkThemeSupport
+ResponsiveLayout
uses DataAccessService
delegates to DataAccessService
uses InteractionService
delegates to InteractionService
«submodule»
ServicesSubmodule
+InteractionService
+DataAccessService
+SingletonServices
+ThreadSafeOperations